Experiment 3

Binary Classification and Decision Boundary Analysis using Logistic Regression

Aim: To implement and analyze a logistic regression model for binary classification and evaluate its predictive performance using classification performance measures.

Theory:

1. Classification vs. Regression

In supervised machine learning, models are trained using datasets in which each input sample is associated with a known target output. Depending on the nature of this target variable, supervised learning problems are broadly categorized into **regression** and **classification** tasks.

- **Regression** involves predicting a continuous numerical value. For example, estimating the price of a house based on its area, location, and number of rooms is a regression problem. Algorithms such as **linear regression** are commonly used for this type of task.
- **Classification** involves predicting a discrete class label or category. For instance, determining whether a patient has dengue (Yes/No) based on clinical symptoms and test results is a classification problem. Algorithms such as **logistic regression** are specifically designed for classification tasks.

Understanding the distinction between regression and classification is important because the choice of algorithm depends on the type of output variable being predicted.

Despite containing the word "regression" in its name, logistic regression is a classification algorithm. The name comes from the fact that it models the log-odds (a continuous quantity) as a linear function of the inputs, and then transforms this value into a probability using the logistic (sigmoid) function.

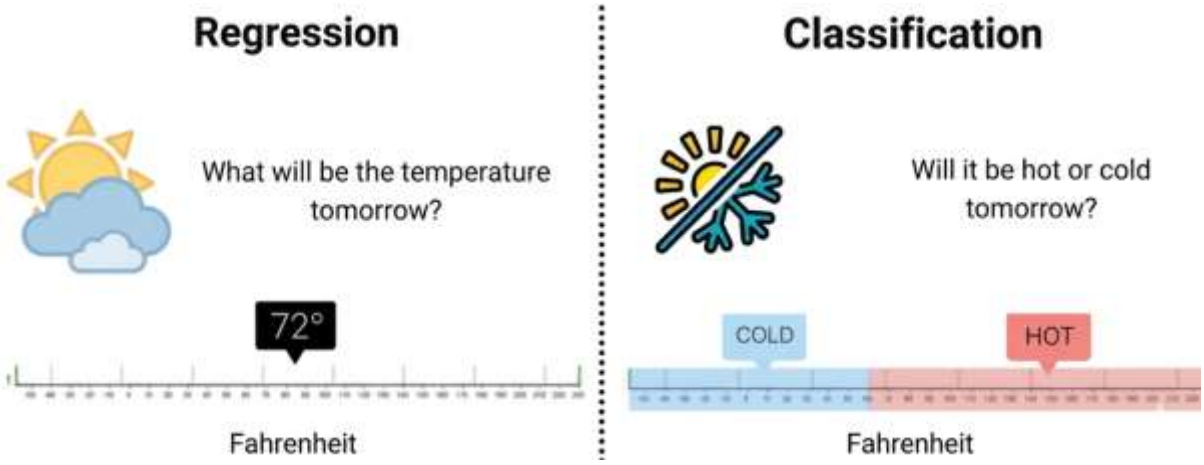


Figure 1: Conceptual Difference Between Regression and Classification in Machine Learning

The figure 1 compares regression and classification tasks in machine learning using a temperature example. Regression predicts an exact numerical value (72°F), while classification predicts a category such as cold or hot based on a temperature range.

2. Introduction to Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems, where the objective is to predict the probability that a given input belongs to a particular class. Unlike linear regression, which predicts continuous numerical values, logistic regression is designed to produce outputs that represent probabilities between 0 and 1. It achieves this by first computing a linear combination of the input features and then transforming this value using a nonlinear function so that the output can be interpreted as the likelihood of belonging to a specific class. Logistic regression is most commonly applied to binary classification tasks, such as determining whether an email is spam or not, whether a patient has a disease or not, or whether a transaction is fraudulent or legitimate. Due to its simplicity, interpretability, and ability to model class probabilities, logistic regression is widely used as a baseline classification method in many practical machine learning applications.

3. Linear Combination in Logistic Regression

In logistic regression, the model first computes a linear combination of the input features, exactly as in linear regression:

$$\boldsymbol{z} = \boldsymbol{\beta^0} + \boldsymbol{\beta^1 x^1} + \boldsymbol{\beta^2 x^2} + \ldots + \boldsymbol{\beta_n x_n} = \boldsymbol{w} \cdot \boldsymbol{x} + \boldsymbol{b}$$

where:

- $\boldsymbol{x^1}, \boldsymbol{x^2}, \ldots, \boldsymbol{x_n}$ are the input features (independent variables). In the dengue classification experiment, these include clinical attributes such as Age, Fever Days, Hematocrit, WBC count, and binary symptom indicators like Headache, Eye Pain, and Muscle Pain.
- $\boldsymbol{\beta_1}, \boldsymbol{\beta_2}, \ldots, \boldsymbol{\beta_n}$ (equivalently $\boldsymbol{w}$, the weight vector) are the model coefficients (parameters) that the algorithm learns during training. Each coefficient quantifies the contribution of its corresponding feature to the prediction.
- $\boldsymbol{\beta_0}$ (equivalently $\boldsymbol{b}$, the bias or intercept term) allows the decision boundary to shift independently of the feature values. Without a bias, the model would be forced to pass through the origin in feature space.

The quantity **z** is called the **log-odds** or **logit** of the positive class, because as we shall see, it equals the logarithm of the ratio of the probability of the positive class to the probability of the negative class.

4. The Sigmoid (Logistic) Function

The computed value **z** can be any real number, ranging from $-\infty \; to \; +\infty$. To convert it into a probability, logistic regression applies the **sigmoid function** (also called the logistic function):

$$\boldsymbol{p} = \boldsymbol{\sigma(z)} = \frac{\mathbf{1}}{\mathbf{1} + \boldsymbol{e^{-z}}}$$

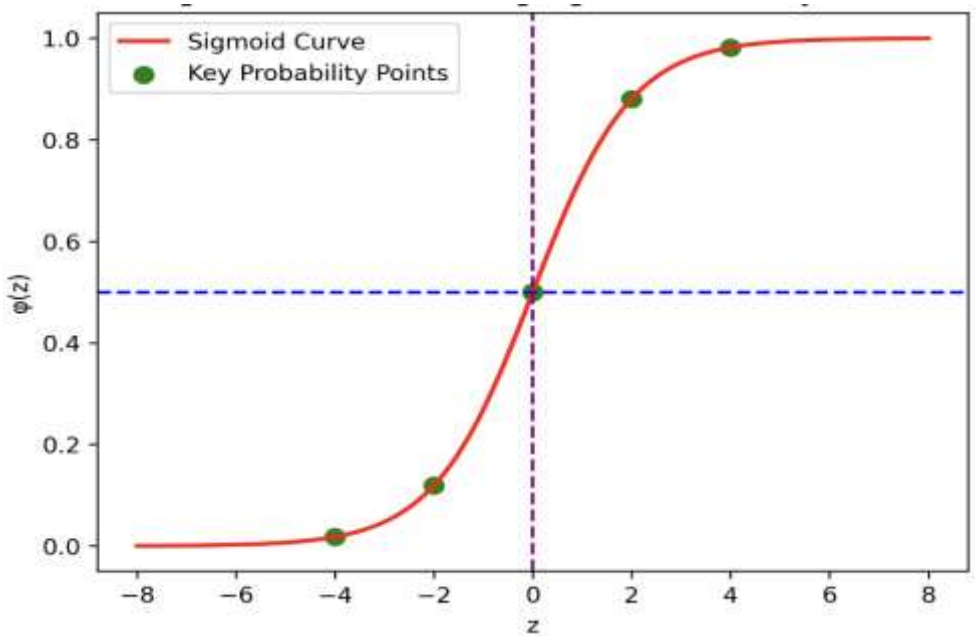


Figure 2: Sigmoid Function Curve Showing Probability Transformation

The figure 2 shows the sigmoid (logistic) function, which converts a linear input value into a probability between 0 and 1 in logistic regression. The S-shaped curve has a midpoint at z = 0 (probability = 0.5), which acts as the decision boundary for binary classification.

Key properties of the sigmoid function:

1. **Bounded output**: $\sigma(z)$ always lies strictly between 0 and 1, so the output is directly interpretable as a probability.

2. **Monotonically increasing**: As z increases, $\sigma(z)$ increases. This means that higher values of z correspond to higher probabilities of belonging to the positive class.

3. **Symmetric around z = 0**: $\sigma(0) = 0.5$, which provides a natural decision boundary. When $z > 0$, the probability exceeds 0.5 (predict positive); when $z < 0$, the probability is below 0.5 (predict negative).

4. **Smooth and differentiable**: The sigmoid has a continuous derivative at every point, which is essential for gradient-based optimisation. Its derivative has an elegant form:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)).$$

5. **Asymptotic behaviour**: $As\ z = +\infty, \sigma(z) = 1;\ as\ z = -\infty, \sigma(z) = 0$. The function never actually reaches 0 or 1.

5. The Logit (Log-Odds) Interpretation

The inverse of the sigmoid function is called the **logit function**. If we denote the probability of the positive class as p, then:

$$\boldsymbol{logit(p) = \log\left(\frac{p}{1-p}\right) = z = w \cdot x + b}$$

The quantity $\frac{p}{1-p}$ is known as the **odds** . The ratio of the probability of the event occurring to the probability of it not occurring. The logit is the natural logarithm of the odds. For example, if p = 0.8,

the odds are 0.8/0.2 = 4 (i.e., the event is 4 times more likely to occur than not), and the logit is log(4) ≈ 1.386.

In logistic regression, the logit is modelled as a linear function of the features. This means that each unit increase in a feature $\boldsymbol{x}_j$ changes the log-odds by $\boldsymbol{\beta}_j$, or equivalently, multiplies the odds by $\boldsymbol{e}^{\boldsymbol{\beta}_j}$. This property makes logistic regression coefficients directly interpretable: a positive coefficient increases the odds of the positive class, while a negative coefficient decreases them.

6. Output and Prediction

The output $\boldsymbol{p}$ represents the probability that the input instance belongs to the positive class (e.g., dengue-positive). To convert this probability into a class label, a **decision threshold** is applied:

Prediction:

- $\boldsymbol{Class\ 1}\ (Positive)\ if\ \boldsymbol{p} \geq \boldsymbol{threshold}$
- $\boldsymbol{Class\ 0}\ (Negative)\ if\ \boldsymbol{p} < \boldsymbol{threshold}$

The default threshold is 0.5 because it corresponds to the symmetry point of the sigmoid $(\sigma(0) = 0.5)$ and treats both classes equally. However, in practice, the threshold can be adjusted based on the application:

- In medical diagnosis (such as dengue detection), a **lower threshold** (e.g., 0.3) may be chosen to increase sensitivity (recall), ensuring that fewer positive cases are missed, even at the cost of more false alarms.
- In spam detection, a **higher threshold** (e.g., 0.7) may be preferred to increase precision, ensuring that legitimate emails are not incorrectly classified as spam.

7. Linear vs Logistic Regression

Aspect	Linear Regression	Logistic Regression
Purpose	Used to predict continuous numerical values	Used to predict categorical outcomes (usually binary)
Type of Problem	Regression	Classification
Output	Continuous numeric value	Probability between **0 and 1**, then converted to class label
Example	Predict house price from area	Predict whether a patient has dengue (Yes/No)
Mathematical Model	$y = w \cdot x + b$	$p = \frac{1}{1 + e^{-(w \cdot x + b)}}$
Activation Function	No activation function	Sigmoid (logistic) function
Output Range	$-\infty\ to\ +\infty$	$0\ to\ 1$

Decision Boundary	Not required	Uses threshold (usually 0.5)
Loss Function	Mean Squared Error (MSE)	$\frac{Binary\ Cross\ Entropy}{Log\ Loss}$
Interpretation	Predicts exact numeric value	Predicts probability of belonging to a class
Evaluation Metrics	MSE, RMSE, R^2	Accuracy, Precision, Recall, F1-Score, ROC-AUC
Graph Shape	Straight line	S-shaped sigmoid curve
Typical Applications	House price prediction, stock prediction	Disease detection, spam detection, fraud detection

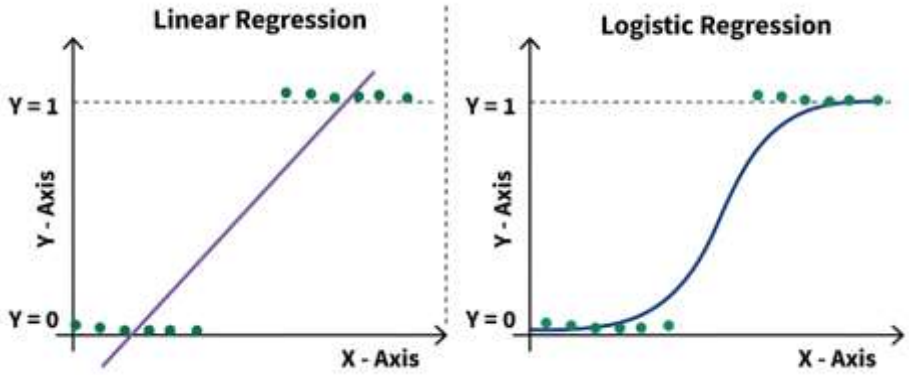


Figure 3: Comparison of Linear Regression and Logistic Regression Decision Behavior

The figure 3 compares linear regression and logistic regression models for binary outcomes. Linear regression fits a straight line that can produce values beyond 0 and 1, while logistic regression uses a sigmoid curve to constrain predictions between 0 and 1 for classification.

8. Loss Function (Binary Cross-Entropy)

Logistic regression uses the binary cross-entropy loss function (also called log loss), which is derived from the principle of maximum likelihood estimation:

$$\boldsymbol{L = -\left(\frac{1}{N}\right)\Sigma\,[y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)]}$$

where:

- $\boldsymbol{N}$ is the number of training samples
- $\boldsymbol{y_i}$ is the actual label (0 or 1) of the i-th sample
- $\boldsymbol{p_i}$ is the predicted probability for the i-th sample

Understanding the loss function intuitively:

- When $y_i = 1$ (actual positive), the loss for that sample is $-log(p_i)$. If the model predicts p close to 1, $-log(1) \approx 0$ (low loss). If it predicts p close to $0, -log(0) = \infty$ (very high loss, strong penalty).

- When $y_i = 0$ (actual negative), the loss is $-log(1 - p_i)$. If the model predicts p close to 0, the loss is low. If it predicts p close to 1, the loss is very high.

This loss function is convex, guaranteeing that gradient descent will converge to the global minimum.

9. Gradient Descent

The model parameters (weights and bias) are updated iteratively using **gradient descent**:

$$\boldsymbol{\beta_j} = \boldsymbol{\beta_j} - \boldsymbol{\alpha} \cdot \left(\frac{\partial \boldsymbol{L}}{\partial \boldsymbol{\beta_j}}\right)$$

where **α** is the learning rate (a hyperparameter controlling the step size). The gradient of the log loss with respect to each weight is:

$$\frac{\partial \boldsymbol{L}}{\partial \boldsymbol{\beta_j}} = \left(\frac{\mathbf{1}}{\boldsymbol{N}}\right) \boldsymbol{\Sigma} \left(\boldsymbol{p_i} - \boldsymbol{y_i}\right) \cdot \boldsymbol{x_{ij}}$$

This gradient has a clean and intuitive form: it is the average of the prediction error $(p_i - y_i)$ weighted by the feature value x_{ij}. The weights are adjusted in the direction that reduces the prediction error.

10. Regularization

When the number of features is large or the features are correlated, the learned weights can become very large, causing the model to overfit the training data. Regularization combats this by adding a penalty term to the loss function that discourages large weight values.

L1 Regularization (Lasso)

$$\boldsymbol{L_{regularized}} = \boldsymbol{L} + \boldsymbol{\lambda} \cdot \boldsymbol{\Sigma} \left|\boldsymbol{\beta_j}\right|$$

L1 regularization adds the sum of absolute weights. It can drive some coefficients exactly to zero, effectively performing feature selection.

L2 Regularization (Ridge)

$$\boldsymbol{L_{regularized}} = \boldsymbol{L} + \boldsymbol{\lambda} \cdot \boldsymbol{\Sigma}\, \boldsymbol{\beta_j}^2$$

L2 regularization adds the sum of squared weights to the loss. It shrinks all coefficients toward zero but does not set any coefficient exactly to zero. In scikit-learn, the regularization strength is controlled by the parameter $\boldsymbol{C} = \frac{\mathbf{1}}{\boldsymbol{\lambda}}$: a smaller C means stronger regularization.

11. Training Algorithm

Step 1: Initialise Parameters

- Set all weights $\beta_1, \beta_2, \ldots, \beta_n$ and bias β_0 to small random values or zeros.

Step 2: Compute the Linear Combination

- For each training example, calculate $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$.

Step 3: Apply the Sigmoid Function

- Compute the predicted probability: $p = \frac{1}{1 + e^{-z}}$.
- The result is always between 0 and 1.

Step 4: Compute the Loss

- Calculate the binary cross-entropy loss over all training examples:

$$L = -\left(\frac{1}{N}\right) \Sigma \left[y_{\mathrm{i}} \log(p_{\mathrm{i}}) + (1 - y_{\mathrm{i}}) \log(1 - p_{\mathrm{i}})\right]$$

Step 5: Compute the Gradient

- For each weight: $\frac{\partial L}{\partial \beta_{\mathrm{j}}} = \left(\frac{1}{N}\right) \Sigma \left(p_{\mathrm{i}} - y_{\mathrm{i}}\right) \cdot x_{\mathrm{ij}}$
- For the bias: $\frac{\partial L}{\partial \beta^{0}} = \left(\frac{1}{N}\right) \Sigma \left(p_{\mathrm{i}} - y_{\mathrm{i}}\right)$

Step 6: Update Parameters

- $\beta_{\mathrm{j}} = \left(\beta_{\mathrm{j}} - \alpha\right) \cdot \frac{\partial L}{\partial \beta_{\mathrm{j}}}$ (for all j)
- α is the learning rate hyperparameter

Step 7: Repeat Until Convergence

- Repeat Steps 2–6 until the change in loss between successive iterations falls below a tolerance threshold, or a maximum number of iterations is reached.

Step 8: Prediction

- For a new input, calculate p using the learned weights.
- Apply the decision threshold (default 0.5):
- $If\ p \geq 0.5, predict\ Class\ 1$
- $If\ p < 0.5, predict\ Class\ 0$

12. Evaluation Metrics for Binary Classification

After training the model, its performance must be evaluated on unseen test data. Several metrics are used, each capturing a different aspect of classification quality.

12.1 Confusion Matrix

The confusion matrix is a 2 × 2 table that summarises the four possible outcomes of binary classification:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** The model correctly predicts the positive class.
- **True Negative (TN):** The model correctly predicts the negative class.
- **False Positive (FP):** The model incorrectly predicts the positive class when the actual class is negative (Type I error).

- **False Negative (FN):** The model incorrectly predicts the negative class when the actual class is positive (Type II error).

12.2 Accuracy

$$\boldsymbol{Accuracy} = \frac{(\boldsymbol{TP} + \boldsymbol{TN})}{(\boldsymbol{TP} + \boldsymbol{TN} + \boldsymbol{FP} + \boldsymbol{FN})}$$

Accuracy measures the overall proportion of correct predictions. It is useful when classes are balanced but can be misleading for imbalanced datasets.

12.3 Precision

$$\boldsymbol{Precision} = \frac{\boldsymbol{TP}}{(\boldsymbol{TP} + \boldsymbol{FP})}$$

Precision measures the proportion of predicted positive instances that are actually positive. A high precision value indicates that the model produces fewer false positive predictions.

12.4 Recall (Sensitivity)

$$\boldsymbol{Recall} = \frac{\boldsymbol{TP}}{(\boldsymbol{TP} + \boldsymbol{FN})}$$

Recall measures the proportion of actual positive instances that are correctly identified by the model. A high recall value indicates that the model successfully detects most of the positive cases and produces fewer false negatives.

12.5 F1-Score

$$\boldsymbol{F1} = \boldsymbol{2} \times \frac{(\boldsymbol{Precision} \times \boldsymbol{Recall})}{(\boldsymbol{Precision} + \boldsymbol{Recall})} = \frac{\boldsymbol{2TP}}{(\boldsymbol{2TP} + \boldsymbol{FP} + \boldsymbol{FN})}$$

The F1-Score is the harmonic mean of precision and recall. It provides a single metric that balances both concerns and is particularly useful when the class distribution is uneven.

12.6 Specificity

$$\boldsymbol{Specificity} = \frac{\boldsymbol{TN}}{(\boldsymbol{TN} + \boldsymbol{FP})}$$

Specificity measures the proportion of actual negatives that are correctly identified. It is the complement of the false positive rate.

12.7 ROC Curve and AUC

The **Receiver Operating Characteristic (ROC) curve** plots the True Positive Rate (Recall) against the False Positive Rate $(1 - Specificity)$ at various threshold settings. A model that perfectly separates the two classes produces a curve that passes through the top-left corner (TPR = 1, FPR = 0).

The **Area Under the ROC Curve (AUC)** summarises the overall discriminative ability of the model into a single number:

- $AUC = 1.0$ $(perfect\ classifier)$
- $AUC = 0.5$ $(no\ better\ than\ random\ guessing)$
- $AUC < 0.5$ ($worse\ than\ random$ $(labels\ may\ be\ inverted)$)

In this experiment, the model achieved an AUC of 0.998, indicating near-perfect separation between the two classes.

13. Merits of Logistic Regression

- **Interpretability**: Each coefficient directly quantifies the effect of its feature on the log-odds of the positive class, making the model easy to explain to domain experts and stakeholders.
- **Computational efficiency**: Training requires only convex optimisation, which converges reliably even on large datasets. The algorithm has no expensive operations like matrix inversion of the full feature space.
- **Probabilistic output**: Unlike algorithms that output only class labels, logistic regression provides calibrated probability estimates, enabling flexible threshold tuning for different application requirements.
- **Low risk of overfitting**: With appropriate regularization, logistic regression generalises well even with moderate amounts of training data.
- **Strong baseline**: In practice, logistic regression frequently matches or exceeds the performance of more complex models on linearly separable or moderately nonlinear problems, making it an essential first model to try.

14. Demerits of Logistic Regression

- **Linear decision boundary**: Logistic regression assumes that the log-odds of the outcome are a linear function of the features. It cannot capture complex nonlinear relationships unless feature engineering (e.g., polynomial features, interaction terms) is applied manually.
- **Sensitivity to outliers**: Extreme values in the feature space can disproportionately influence the learned coefficients and shift the decision boundary.
- **Multicollinearity issues**: When input features are highly correlated, the coefficient estimates become unstable and difficult to interpret, even though the overall predictions may remain reasonable.
- **Not suitable for multi-modal class distributions**: When the positive and negative classes form complex, non-convex regions in feature space, logistic regression will underperform compared to tree-based or kernel-based methods.

Logistic Regression - Practical Implementation

Step 1: Importing Libraries

 # Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)
from ipywidgets import interact, Dropdown
print("Libraries Imported");
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

 # Read and store the Dengue dataset

```
data = pd.read_csv('Dengue_dataset.csv')
print("Dataset reading complete")
```

Output:

```
Dataset reading complete
```

Step 3: Data Analysis

 # Display the first 5 rows of the dataset

```
data.head()
```

Output:

```
index  Age  Fever Days  Platelets  Hematocrit   WBC   Headache  EyePain  Muscle Pain  Joint
Pain  Rash  Nausea  Vomiting  Abdominal Pain  Bleeding  Lethargy  Dengue
0       61   6           56777      49.49572857  3205  0         1        0            0
0     1       1         0               0         0         1
1       24   7           61250      42.41093608  3738  1         0        0            0
1     0       0         1               1         0         1
2       70   7           88034      49.66143051  3052  1         0        1            0
0     1       0         0               0         0         1
3       30   6           55130      50.20159282  2713  1         0        0            1
0     0       1         1               0         0         1
4       33   3           64346      43.46980401  4888  1         1        0            1
1     1       1         0               1         0         1
```

Display the last 5 rows of the dataset

```
data.tail()
```

Output:

```
index  Age  Fever Days  Platelets  Hematocrit   WBC   Headache  EyePain  Muscle Pain  Joint
Pain  Rash  Nausea  Vomiting  Abdominal Pain  Bleeding  Lethargy  Dengue
4995    59   2           123791     47.18861033  4239  1         0        0            0
0     0       0         0               0         0         0
4996    11   4           111575     41.84560446  5989  1         0        1            1
1     1       1         0               0         0         0
4997    19   2           128198     40.2842782   4775  1         1        1            0
0     0       0         0               0         1         0
4998    11   0           141619     47.3939162   4625  1         1        0            0
1     1       1         1               0         1         0
4999    69   2           90619      46.47148497  5982  1         0        0            0
1     0       1         0               0         0         0
```

Show statistical summary of numerical columns

```
data.describe()
```

Output:

```
Age      Fever Days  Platelets  Hematocrit  WBC      Headache  Eye Pain  Muscle Pain  Joint Pain
Rash     Nausea  Vomiting  Abdominal Pain  Bleeding  Lethargy  Dengue
count  5000.0  5000.0      5000.0     5000.0      5000.0    5000.0    5000.0    5000.0
5000.0      5000.0  5000.0  5000.0    5000.0          5000.0    5000.0    5000.0
mean   41.85   3.41        141522.48  41.86       6156.58   0.517     0.492     0.508
0.500       0.496   0.501   0.507     0.496           0.226     0.502     0.460
std    18.80   2.05        87116.73   4.81        2594.06   0.500     0.500     0.500
0.500       0.500   0.500   0.500     0.500           0.418     0.500     0.498
min    10.0    0.0         15060.0    32.02       2002.0    0.0       0.0       0.0          0.0
0.0     0.0     0.0       0.0             0.0       0.0       0
25%    26.0    2.0         62365.5    38.43       3848.75   0.0       0.0       0.0          0.0
0.0     0.0     0.0       0.0             0.0       0.0       0
50%    42.0    3.0         130972.0   42.02       6169.0    1.0       0.0       1.0          0.5
0.0     1.0     1.0       0.0             0.0       1.0       0
75%    58.0    5.0         221014.25  44.89       8302.5    1.0       1.0       1.0          1.0
1.0     1.0     1.0       1.0             0.0       1.0       1
max    74.0    7.0         299988.0   51.99       10993.0   1.0       1.0       1.0          1.0
1.0     1.0     1.0       1.0             1.0       1.0       1
```

Display dataset structure and data types

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
0   Age             5000 non-null   int64
1   FeverDays       5000 non-null   int64
2   Platelets       5000 non-null   int64
3   Hematocrit      5000 non-null   float64
4   WBC             5000 non-null   int64
5   Headache        5000 non-null   int64
6   EyePain         5000 non-null   int64
7   MusclePain      5000 non-null   int64
8   JointPain       5000 non-null   int64
9   Rash            5000 non-null   int64
10  Nausea          5000 non-null   int64
11  Vomiting        5000 non-null   int64
12  AbdominalPain   5000 non-null   int64
13  Bleeding        5000 non-null   int64
14  Lethargy        5000 non-null   int64
15  Dengue          5000 non-null   int64
dtypes: float64(1), int64(15)
memory usage: 625.1 KB
```

Check the number of missing values in each column

```
data.isnull().sum()
```

Output:

```
Column          Missing Values
Age             0
FeverDays       0
Platelets       0
Hematocrit      0
WBC             0
Headache        0
EyePain         0
MusclePain      0
JointPain       0
Rash            0
Nausea          0
Vomiting        0
AbdominalPain   0
Bleeding        0
Lethargy        0
Dengue          0
dtype: int64
```

Show the number of rows and columns in the dataset

```
data.shape
```

Output:

```
(5000, 16)
```

Count occurrences of each class in the Dengue column

```
data['Dengue'].value_counts()
```

Output:
```
0    27001    2300Name: Dengue, dtype: int64
```

 # Count frequency of each Platelets value

```
data['Platelets'].value_counts()
```

Output:
```
Platelets
244845    2
32653     2
26618     2
61015     2
58169     2
...
25367     1
72241     1
18385     1
51701     1
70182     1
4961 rows x 1 columns
dtype: int64
```

 # Count frequency of each WBC value

```
data['WBC'].value_counts()
```

Output:
```
WBC
6916     6
6573     5
9771     5
2521     5
7405     5
...
3987     1
4247     1
5815     1
3598     1
5860     1
3781 rows x 1 columns
dtype: int64
```

Step 4: Data Preprocessing

 # Encode Dengue column by flipping values (0 -> 1, 1 -> 0)

```
data = data.replace({'Dengue':{0:1, 1:0}})
data.head()
```

Output:

```
Dataset Preview:
index  Age  FeverDays  Platelets  Hematocrit  WBC      Headache  EyePain  MusclePain  JointPain
Rash  Nausea  Vomiting  AbdominalPain  Bleeding  Lethargy  Dengue
0       61   6          56777       49.4957     3205  0         1        0           0          0
1        1         0              0         0         0
1       24   7          61250       42.4109     3738  1         0        0           0          1
0        0         1              1         0         0
2       70   7          88034       49.6614     3052  1         0        1           0          0
1        0         0              0         0         0
3       30   6          55130       50.2016     2713  1         0        0           1          0
0        1         1              0         0         0
4       33   3          64346       43.4698     4888  1         1        0           1          1
1        1         0              1         0         0
```

Encode Headache column by flipping values (0 -> 1, 1 -> 0)

```
data = data.replace({'Headache':{0:1, 1:0}})
data.head()
```

Output:

```
Dataset Preview:
index  Age  FeverDays  Platelets  Hematocrit  WBC      Headache  EyePain  MusclePain  JointPain
Rash  Nausea  Vomiting  AbdominalPain  Bleeding  Lethargy  Dengue
0       61   6          56777       49.4957     3205  1         1        0           0          0
1        1         0              0         0         0
1       24   7          61250       42.4109     3738  0         0        0           0          1
0        0         1              1         0         0
2       70   7          88034       49.6614     3052  0         0        1           0          0
1        0         0              0         0         0
3       30   6          55130       50.2016     2713  0         0        0           1          0
0        1         1              0         0         0
4       33   3          64346       43.4698     4888  0         1        0           1          1
1        1         0              1         0         0
```

Encode Rash column by flipping values (0 -> 1, 1 -> 0) and verify changes

```
data = data.replace({'Rash':{0:1, 1:0}})
print("Encoding complete. Checked head:")
data.head()
```

Output:

```
Encoding complete. Checked head:
Dataset Preview:
index  Age  FeverDays  Platelets  Hematocrit  WBC      Headache  EyePain  MusclePain  JointPain
Rash  Nausea  Vomiting  AbdominalPain  Bleeding  Lethargy  Dengue
0       61   6          56777       49.4957     3205  1         1        0           0          1
1        1         0              0         0         0
1       24   7          61250       42.4109     3738  0         0        0           0          0
0        0         1              1         0         0
2       70   7          88034       49.6614     3052  0         0        1           0          1
1        0         0              0         0         0
3       30   6          55130       50.2016     2713  0         0        0           1          1
0        1         1              0         0         0
4       33   3          64346       43.4698     4888  0         1        0           1          0
1        1         0              1         0         0
```

Step 5: Model Training

 # Set the target variable, separate features and labels, and split the dataset into 80% training and 20% testing data

```
target_col = "Dengue"
print("Target column set to:", target_col)
X = data.drop(columns=['Platelets', target_col])
y = data[target_col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42,
    stratify=y)
print(f"Split: 80% Train ({len(X_train)}) / 20% Test ({len(X_test)}) samples")
```

Output:

```
Target column set to: Dengue
Split: 80% Train (4000) / 20% Test (1000) samples
```

 # Display the feature matrix

```
X
```

Output:

```
Feature Matrix (X) Preview:
Age  FeverDays  Platelets  Hematocrit  WBC      Headache  EyePain  MusclePain  JointPain  Rash
Nausea  Vomiting  AbdominalPain  Bleeding  Lethargy
0      61    6           56777       49.495729    3205   0          1         0            0           0
1         1          0               0           0
1      24    7           61250       42.410936    3738   1          0         0            0           1
0         0          1               1           0
2      70    7           88034       49.661431    3052   1          0         1            0           0
1         0          0               0           0
3      30    6           55130       50.201593    2713   1          0         0            1           0
0         1          1               0           0
4      33    3           64346       43.469804    4888   1          1         0            1           1
1         1          0               1           0
...    ...   ...         ...         ...          ...    ...        ...       ...          ...
...    ...      ...        ...              ...        ...
4995   59    2           123791      47.188610    4239   1          0         0            0           0
0         0          0               0           0
4996   11    4           111575      41.845604    5989   1          0         1            1           1
1         1          0               0           0
4997   19    2           128198      40.284278    4775   1          1         1            0           0
0         0          0               0           1
4998   11    0           141619      47.393916    4625   1          1         0            0           1
1         1          1               0           1
4999   69    2           90619       46.471485    5982   1          0         0            0           1
0         1          0               0           0
5000 rows x 15 columns
```

 # Display the first few values of the target variable

```
y.head()
```

Output:

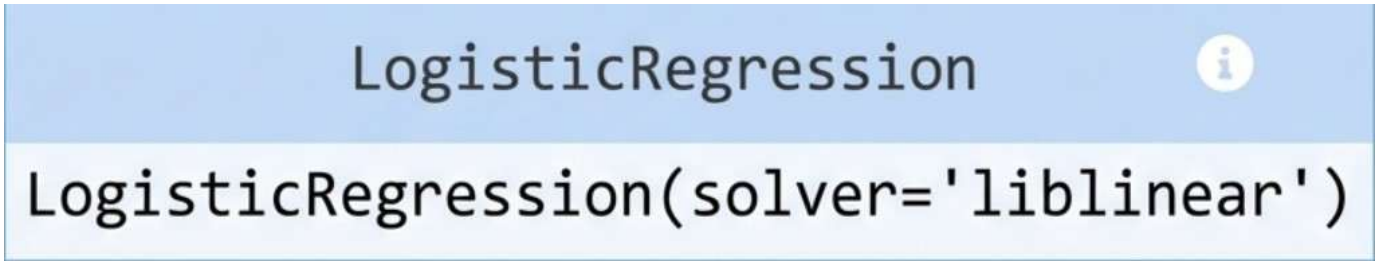
```
Target Vector (y) Preview:
Dengue
0  0
1  0
2  0
3  0
4  0
Name: Dengue, dtype: int64
```

 # Initialize and train a Logistic Regression model with L2 regularization on the training data

```
model = LogisticRegression(C=1.0, penalty='l2', solver='liblinear')
model.fit(X_train, Y_train)
print("Model trained successfully")
```

Output:

```
Model trained successfully
```



Step 6: Model Evaluation

 # Preparing Feature and Target Data for Logistic Regression Analysis

```
numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
feature_names = numeric_cols
X_num = X[feature_names]
y_arr = np.array(y)
print(f"Feature-target data ready -> {len(feature_names)} numeric features selected for
    modeling")
```

Output:

```
Feature-target data ready -> 14 numeric features selected for modeling
```

 # Fits a logistic regression model on a single feature and visualizes the resulting sigmoid probability curve against th

```
def show_1d_sigmoid(f):
    X,y = X_num[[f]].values, y_arr; c = LogisticRegression(max_iter=5000).fit(X,y)
    a,b = X.min(),X.max();
    a,b = (a-1,b+1) if a==b else (a,b); g= np.linspace(a,b,300)[:,None]; p =
    c.predict_proba(g)[:,1]
    plt.scatter(X[y==0],y[y==0]); plt.scatter(X[y==1],y[y==1]); plt.plot(g,p);
    plt.xlabel(f);
    plt.ylabel("Probability of Dengue");
    plt.ylim(-.1,1.1); plt.grid(1); plt.title(f"Sigmoid using {f}");
    plt.show()

print("Function defined: show_1d_sigmoid")
```

Output:

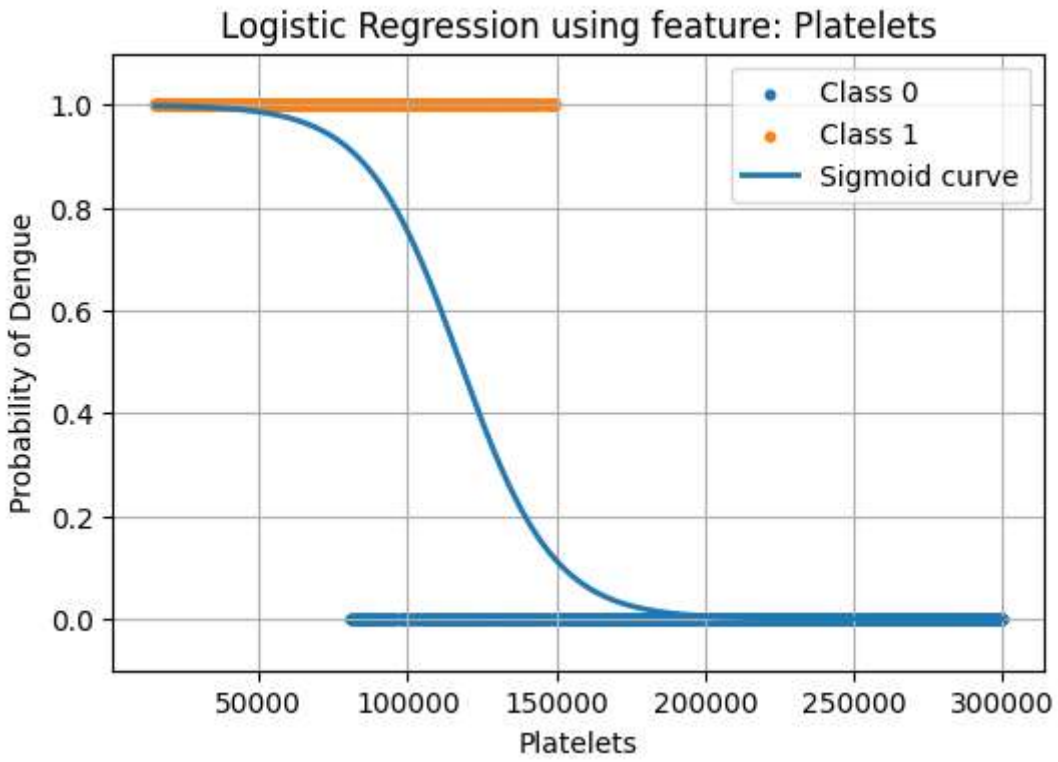
```
Function defined: show_1d_sigmoid
```

Interactive dropdown over all features

```
interact(
    show_1d_sigmoid,
    feat=Dropdown(options=feature_names, description="Feature")
)
```

Output:

```
Feature
Age
FeverDays
Platelets
Hematocrit
WBC
MusclePain
JointPain
Nausea
Vomiting
AbdominalPain
Bleeding
Lethargy
Rash
EyePain
Headache
```



Training Performance

```
Y_train_pred = model.predict(X_train)
print(" Training Performance ")
print(f"Accuracy : {accuracy_score(y_train, Y_train_pred):.4f}")
print(f"Precision: {precision_score(y_train, Y_train_pred):.4f}")
print(f"Recall   : {recall_score(y_train, Y_train_pred):.4f}")
print(f"F1 Score : {f1_score(y_train, Y_train_pred):.4f}")
```

Output:

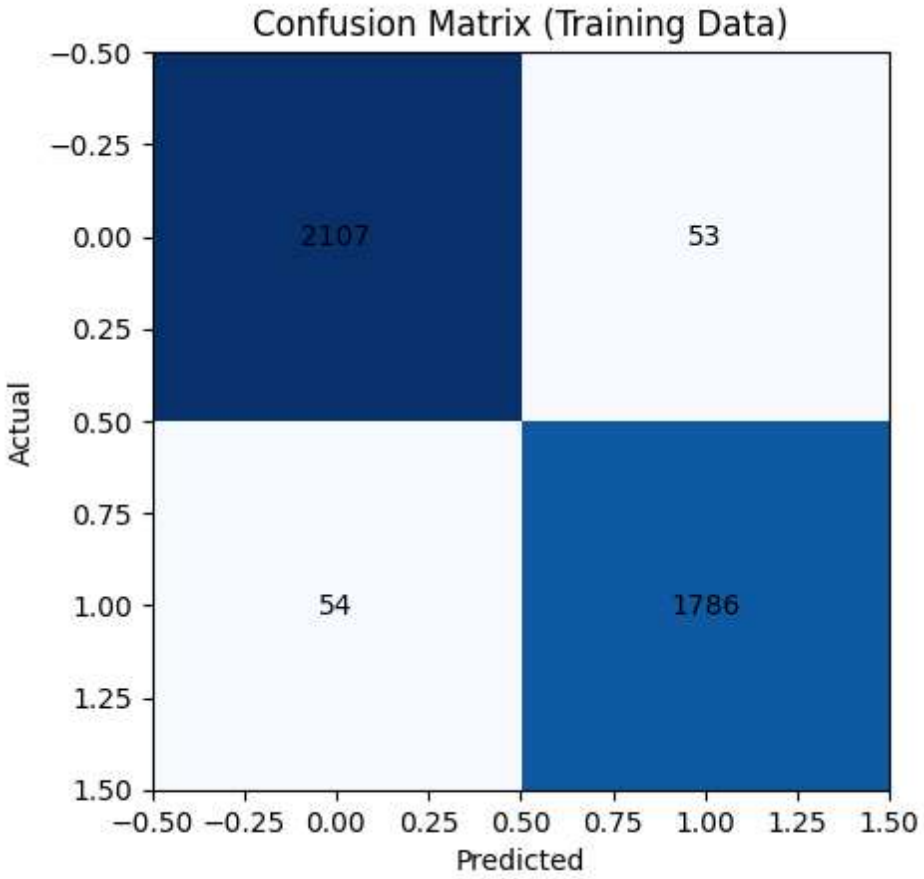
```
Training Performance
Accuracy : 0.9732
Precision: 0.9712
Recall   : 0.9707
F1 Score : 0.9709
```

Confusion Matrix for training data

```
cm_train = confusion_matrix(y_train, Y_train_pred)
sns.heatmap(cm_train, cmap="Blues")
plt.title("Confusion Matrix (Training Data)")
for i in range(len(cm_train)):
    for j in range(len(cm_train)):
        plt.text(j, i, cm_train[i][j], ha='center', va='center')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
Confusion Matrix (Training Data)
```



Testing Performance

```
Y_test_pred = model.predict(X_test)
print("Testing Performance")
print(f"Accuracy : {accuracy_score(y_test, Y_test_pred):.4f}")
print(f"Precision: {precision_score(y_test, Y_test_pred):.4f}")
print(f"Recall   : {recall_score(y_test, Y_test_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, Y_test_pred):.4f}")
```

Output:

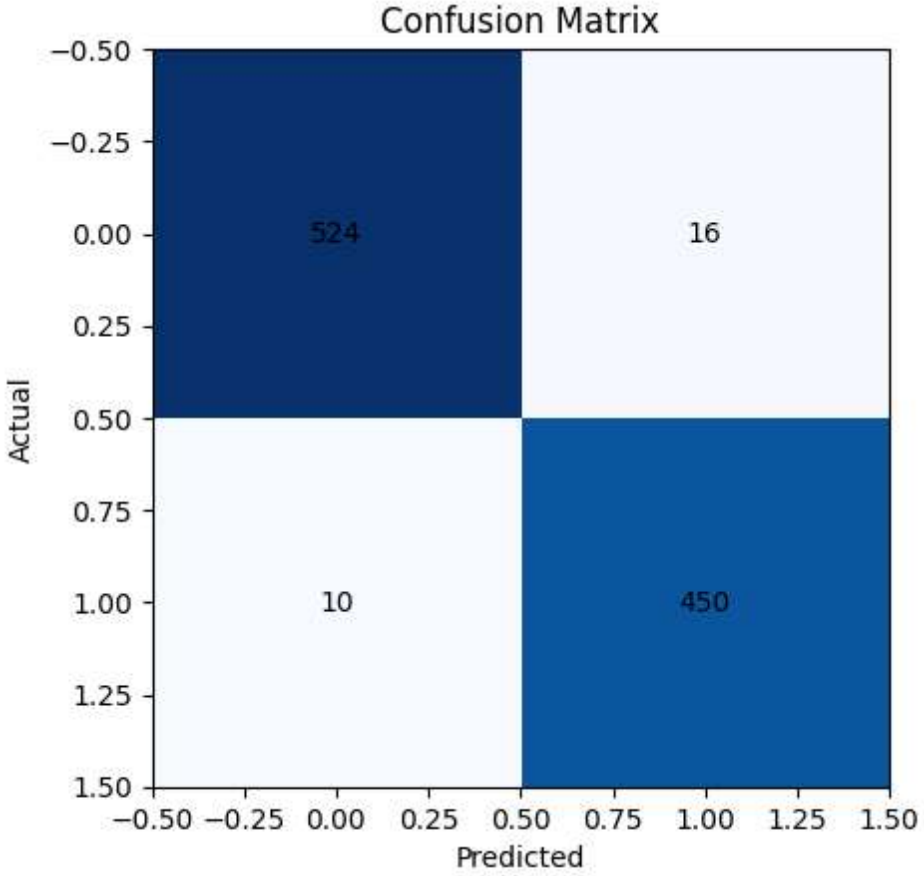
```
Testing Performance
Accuracy : 0.9740
Precision: 0.9657
Recall   : 0.9783
F1 Score : 0.9719
```

Confusion Matrix for testing data

```
cm_test = confusion_matrix(y_test, Y_test_pred)
sns.heatmap(cm_test, annot=True, cmap="Blues", fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix for Testing Data")
plt.show()
```

Output:

```
Confusion Matrix (Testing Data)
```



 # Calculates predicted probabilities to compute ROC curve metrics and the AUC score.

```
Y_proba = model.predict_proba(X_test)[:,1]
fpr, tpr, th = roc_curve(Y_test, Y_proba)
roc_auc = auc(fpr, tpr)
print(f"ROC metrics calculated. AUC = {roc_auc:.3f}")
```

Output:

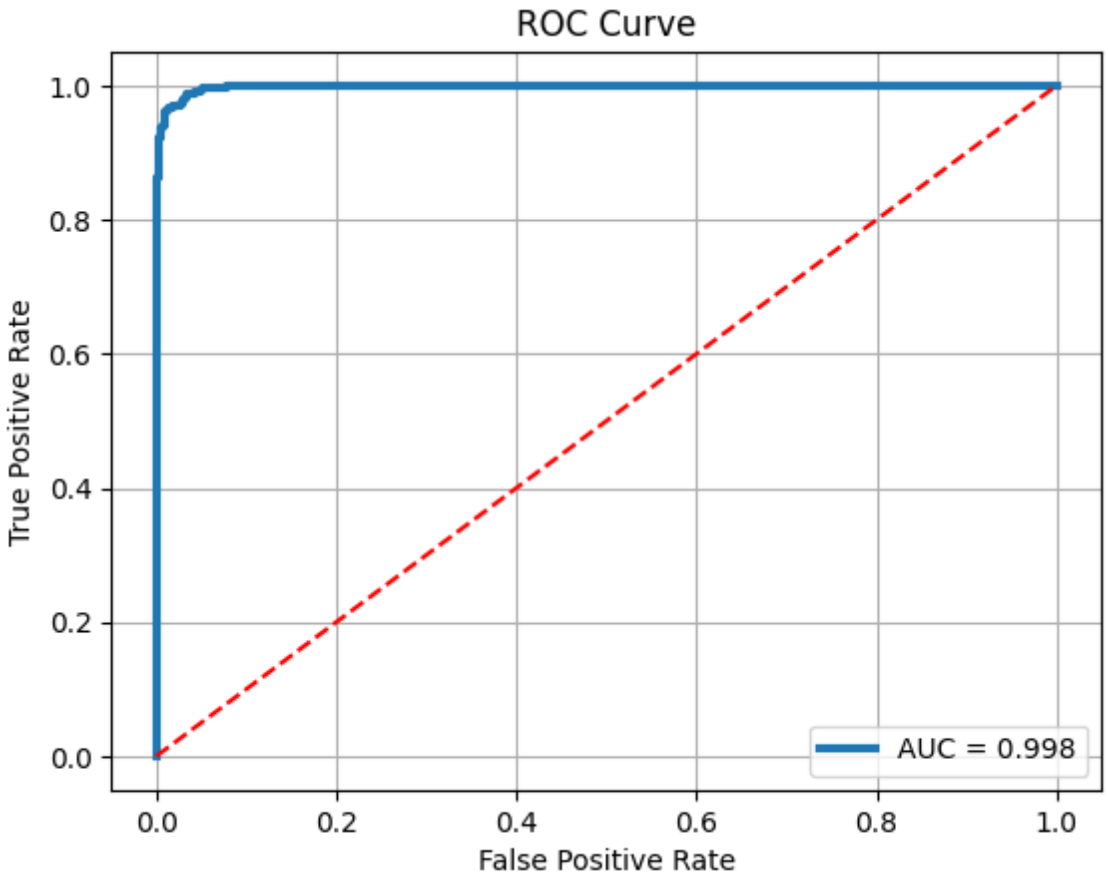
```
ROC metrics calculated. AUC = 0.998
```

 # Plots the ROC curve including the AUC score and a random classifier baseline.

```
plt.plot(fpr, tpr, linewidth=3, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1], [0,1], 'r--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid()
plt.show()
```

Output:

```
ROC Curve
```



Random Test Prediction

```
test_samples=data.sample(5)
display(test_samples)
sample = X_test.sample(1)
actual = Y_test.loc[sample.index].values[0]
pred = model.predict(sample)[0]
prob = model.predict_proba(sample)[0][1]
print("------ Random Sample Test-------")
print(sample)
print("Actual Dengue:", actual)
print("Predicted Dengue:", pred)
print("Probability:", prob)
```

Output:

```
Select a row to see prediction:
Index  Age  FeverDays  Hematocrit  WBC  Headache  EyePain  MusclePain  JointPain  Rash  Nausea
Vomiting  AbdominalPain  Bleeding  Lethargy
```